

The JavaScript language

JS

Day 1

Java Scripts Fundamentals

Code structure

Variables

Data types

Basic operators, maths

Comparisons

Conditional branching: if, '?'

Logical operators

Nullish coalescing operator '??'

Loops: while and for

The "switch" statement

Code structure

The first thing we'll study is the building blocks of code.

Statements

Statements are syntax constructs and commands that perform actions.

We've already seen a statement, `alert('Hello, world!')`, which shows the message "Hello, world!".

We can have as many statements in our code as we want. Statements can be separated with a semicolon.

For example, here we split "Hello World" into two alerts:

```
alert('Hello'); alert('World');
```

Usually, statements are written on separate lines to make the code more readable:

```
alert('Hello');  
alert('World');
```

Semicolons

A semicolon may be omitted in most cases when a line break exists.

This would also work:

```
alert('Hello')  
alert('World')
```

Here, JavaScript interprets the line break as an "implicit" semicolon. This is called an [automatic semicolon insertion](#).

In most cases, a newline implies a semicolon. But "in most cases" does not mean "always"!

There are cases when a newline does not mean a semicolon. For example:

```
alert(3 +  
1
```

```
+ 2);
```

The code outputs `6` because JavaScript does not insert semicolons here. It is intuitively obvious that if the line ends with a plus `"+"`, then it is an “incomplete expression”, so a semicolon there would be incorrect. And in this case, that works as intended.

But there are situations where JavaScript “fails” to assume a semicolon where it is really needed.

Errors which occur in such cases are quite hard to find and fix.

An example of an error

If you're curious to see a concrete example of such an error, check this code out:

```
alert("Hello");  
  
[1, 2].forEach(alert);
```

No need to think about the meaning of the brackets `[]` and `forEach` yet. We'll study them later. For now, just remember the result of running the code: it shows `Hello`, then `1`, then `2`.

Now let's remove the semicolon after the `alert`:

```
alert("Hello")  
  
[1, 2].forEach(alert);
```

The difference compared to the code above is only one character: the semicolon at the end of the first line is gone.

If we run this code, only the first `Hello` shows (and there's an error, you may need to open the console to see it). There are no numbers any more.

That's because JavaScript does not assume a semicolon before square brackets `[...]`. So, the code in the last example is treated as a single statement.

Here's how the engine sees it:

```
alert("Hello")[1, 2].forEach(alert);
```

Looks weird, right? Such merging in this case is just wrong. We need to put a semicolon after `alert` for the code to work correctly.

This can happen in other situations also.

We recommend putting semicolons between statements even if they are separated by newlines. This rule is widely adopted by the community. Let's note once again – *it is possible* to leave out semicolons most of the time. But it's safer – especially for a beginner – to use them.

Comments

As time goes on, programs become more and more complex. It becomes necessary to add *comments* which describe what the code does and why.

Comments can be put into any place of a script. They don't affect its execution because the engine simply ignores them.

One-line comments start with two forward slash characters `//`.

The rest of the line is a comment. It may occupy a full line of its own or follow a statement.

Like here:

```
// This comment occupies a line of its own  
alert('Hello');  
  
alert('World'); // This comment follows the statement
```

Multiline comments start with a forward slash and an asterisk `/*` and end with an asterisk and a forward slash `*/`.

Like this:

```
/* An example with two messages.  
This is a multiline comment.  
*/  
alert('Hello');  
alert('World');
```

The content of comments is ignored, so if we put code inside `/* ... */`, it won't execute.

Sometimes it can be handy to temporarily disable a part of code:

```
/* Commenting out the code  
alert('Hello');  
*/  
alert('World');
```

Use hotkeys!

In most editors, a line of code can be commented out by pressing the `Ctrl+/*` hotkey for a single-line comment and something like `Ctrl+Shift+/*` – for multiline comments (select a piece of code and press the hotkey). For Mac, try `Cmd` instead of `Ctrl` and `Option` instead of `Shift`.

Nested comments are not supported!

There may not be `/*...*/` inside another `/*...*/`.

Such code will die with an error:

```
/*  
  /* nested comment !?! */  
*/  
alert( 'World' );
```

Please, don't hesitate to comment your code.

Comments increase the overall code footprint, but that's not a problem at all. There are many tools which minify code before publishing to a production server. They remove comments, so they don't appear in the working scripts. Therefore, comments do not have negative effects on production at all.

Later in the tutorial there will be a chapter [Code quality](#) that also explains how to write better comments.

The modern mode, "use strict"

For a long time, JavaScript evolved without compatibility issues. New features were added to the language while old functionality didn't change.

That had the benefit of never breaking existing code. But the downside was that any mistake or an imperfect decision made by JavaScript's creators got stuck in the language forever.

This was the case until 2009 when ECMAScript 5 (ES5) appeared. It added new features to the language and modified some of the existing ones. To keep the old code working, most such modifications are off by default. You need to explicitly enable them with a special directive: `"use strict"`.

"use strict"

The directive looks like a string: `"use strict"` or `'use strict'`. When it is located at the top of a script, the whole script works the "modern" way.

For example:

```
"use strict";
```

```
// this code works the modern way
...
```

Quite soon we're going to learn functions (a way to group commands), so let's note in advance that `"use strict"` can be put at the beginning of a function. Doing that enables strict mode in that function only. But usually people use it for the whole script.

⚠ Ensure that `"use strict"` is at the top

Please make sure that `"use strict"` is at the top of your scripts, otherwise strict mode may not be enabled.

Strict mode isn't enabled here:

```
alert("some code");
// "use strict" below is ignored--it must be at the top

"use strict";

// strict mode is not activated
```

Only comments may appear above `"use strict"`.

⚠ There's no way to cancel `use strict`

There is no directive like `"no use strict"` that reverts the engine to old behavior.

Once we enter strict mode, there's no going back.

Browser console

When you use a [developer console](#) to run code, please note that it doesn't `use strict` by default.

Sometimes, when `use strict` makes a difference, you'll get incorrect results.

So, how to actually `use strict` in the console?

First, you can try to press `Shift+Enter` to input multiple lines, and put `use strict` on top, like this:

```
'use strict'; <Shift+Enter for a newline>
// ...your code
<Enter to run>
```

It works in most browsers, namely Firefox and Chrome.

If it doesn't, e.g. in an old browser, there's an ugly, but reliable way to ensure `use strict`. Put it inside this kind of wrapper:

```
(function() {
  'use strict';

  // ...your code here...
})();
```

Should we `"use strict"`?

The question may sound obvious, but it's not so.

One could recommend to start scripts with `"use strict"` ... But you know what's cool?

Modern JavaScript supports "classes" and "modules" – advanced language structures (we'll surely get to them), that enable `use strict` automatically. So we don't need to add the `"use strict"` directive, if we use them.

So, for now `"use strict";` is a welcome guest at the top of your scripts. Later, when your code is all in classes and modules, you may omit it.

As of now, we've got to know about `use strict` in general.

In the next chapters, as we learn language features, we'll see the differences between the strict and old modes. Luckily, there aren't many and they actually make our lives better.

All examples in this tutorial assume strict mode unless (very rarely) specified otherwise.

Variables

Most of the time, a JavaScript application needs to work with information. Here are two examples:

1. An online shop – the information might include goods being sold and a shopping cart.
2. A chat application – the information might include users, messages, and much more.

Variables are used to store this information.

A variable

A [variable](#) is a “named storage” for data. We can use variables to store goodies, visitors, and other data.

To create a variable in JavaScript, use the `let` keyword.

The statement below creates (in other words: *declares*) a variable with the name “message”:

```
let message;
```

Now, we can put some data into it by using the assignment operator `=`:

```
let message;  
  
message = 'Hello'; // store the string 'Hello' in the variable named message
```

The string is now saved into the memory area associated with the variable. We can access it using the variable name:

```
let message;  
message = 'Hello!';  
  
alert(message); // shows the variable content
```

To be concise, we can combine the variable declaration and assignment into a single line:

```
let message = 'Hello!'; // define the variable and assign the value  
  
alert(message); // Hello!
```

We can also declare multiple variables in one line:

```
let user = 'John', age = 25, message = 'Hello';
```

That might seem shorter, but we don't recommend it. For the sake of better readability, please use a single line per variable.

The multiline variant is a bit longer, but easier to read:

```
let user = 'John';  
let age = 25;  
let message = 'Hello';
```

Some people also define multiple variables in this multiline style:

```
let user = 'John',  
    age = 25,  
    message = 'Hello';
```

...Or even in the “comma-first” style:

```
let user = 'John',
    age = 25,
    message = 'Hello';
```

Technically, all these variants do the same thing. So, it's a matter of personal taste and aesthetics.

i var instead of let

In older scripts, you may also find another keyword: `var` instead of `let`:

```
var message = 'Hello';
```

The `var` keyword is *almost* the same as `let`. It also declares a variable but in a slightly different, “old-school” way.

There are subtle differences between `let` and `var`, but they do not matter to us yet. We'll cover them in detail in the chapter [The old "var"](#).

A real-life analogy

We can easily grasp the concept of a “variable” if we imagine it as a “box” for data, with a uniquely-named sticker on it.

For instance, the variable `message` can be imagined as a box labelled “message” with the value “Hello!” in it:



We can put any value in the box.

We can also change it as many times as we want:

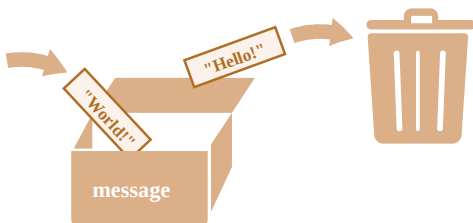
```
let message;

message = 'Hello!';

message = 'World!'; // value changed

alert(message);
```

When the value is changed, the old data is removed from the variable:



We can also declare two variables and copy data from one into the other.

```
let hello = 'Hello world!';

let message;

// copy 'Hello world' from hello into message
message = hello;
```

```
// now two variables hold the same data
alert(hello); // Hello world!
alert(message); // Hello world!
```

⚠ Declaring twice triggers an error

A variable should be declared only once.

A repeated declaration of the same variable is an error:

```
let message = "This";

// repeated 'let' leads to an error
let message = "That"; // SyntaxError: 'message' has already been declared
```

So, we should declare a variable once and then refer to it without `let`.

i Functional languages

It's interesting to note that there exist so-called [pure functional](#) programming languages, such as [Haskell](#), that forbid changing variable values.

In such languages, once the value is stored "in the box", it's there forever. If we need to store something else, the language forces us to create a new box (declare a new variable). We can't reuse the old one.

Though it may seem a little odd at first sight, these languages are quite capable of serious development. More than that, there are areas like parallel computations where this limitation confers certain benefits.

Variable naming

There are two limitations on variable names in JavaScript:

1. The name must contain only letters, digits, or the symbols `$` and `_`.
2. The first character must not be a digit.

Examples of valid names:

```
let userName;
let test123;
```

When the name contains multiple words, [camelCase](#) is commonly used. That is: words go one after another, each word except first starting with a capital letter: `myVeryLongName`.

What's interesting – the dollar sign `'$'` and the underscore `'_'` can also be used in names. They are regular symbols, just like letters, without any special meaning.

These names are valid:

```
let $ = 1; // declared a variable with the name "$"
let _ = 2; // and now a variable with the name "_"

alert($ + _); // 3
```

Examples of incorrect variable names:

```
let 1a; // cannot start with a digit

let my-name; // hyphens '-' aren't allowed in the name
```

i Case matters

Variables named `apple` and `APPLE` are two different variables.

Non-Latin letters are allowed, but not recommended

It is possible to use any language, including Cyrillic letters, Chinese logograms and so on, like this:

```
let имя = '...';  
let 我 = '...';
```

Technically, there is no error here. Such names are allowed, but there is an international convention to use English in variable names. Even if we're writing a small script, it may have a long life ahead. People from other countries may need to read it sometime.

Reserved names

There is a [list of reserved words](#), which cannot be used as variable names because they are used by the language itself.

For example: `let`, `class`, `return`, and `function` are reserved.

The code below gives a syntax error:

```
let let = 5; // can't name a variable "let", error!  
let return = 5; // also can't name it "return", error!
```

An assignment without `use strict`

Normally, we need to define a variable before using it. But in the old times, it was technically possible to create a variable by a mere assignment of the value without using `let`. This still works now if we don't put `use strict` in our scripts to maintain compatibility with old scripts.

```
// note: no "use strict" in this example  
  
num = 5; // the variable "num" is created if it didn't exist  
  
alert(num); // 5
```

This is a bad practice and would cause an error in strict mode:

```
"use strict";  
  
num = 5; // error: num is not defined
```

Constants

To declare a constant (unchanging) variable, use `const` instead of `let`:

```
const myBirthday = '18.04.1982';
```

Variables declared using `const` are called “constants”. They cannot be reassigned. An attempt to do so would cause an error:

```
const myBirthday = '18.04.1982';  
  
myBirthday = '01.01.2001'; // error, can't reassign the constant!
```

When a programmer is sure that a variable will never change, they can declare it with `const` to guarantee and communicate that fact to everyone.

Uppercase constants

There is a widespread practice to use constants as aliases for difficult-to-remember values that are known before execution.

Such constants are named using capital letters and underscores.

For instance, let's make constants for colors in so-called "web" (hexadecimal) format:

```
const COLOR_RED = "#F00";
const COLOR_GREEN = "#0F0";
const COLOR_BLUE = "#00F";
const COLOR_ORANGE = "#FF7F00";

// ...when we need to pick a color
let color = COLOR_ORANGE;
alert(color); // #FF7F00
```

Benefits:

- `COLOR_ORANGE` is much easier to remember than `"#FF7F00"`.
- It is much easier to mistype `"#FF7F00"` than `COLOR_ORANGE`.
- When reading the code, `COLOR_ORANGE` is much more meaningful than `#FF7F00`.

When should we use capitals for a constant and when should we name it normally? Let's make that clear.

Being a "constant" just means that a variable's value never changes. But some constants are known before execution (like a hexadecimal value for red) and some constants are *calculated* in run-time, during the execution, but do not change after their initial assignment.

For instance:

```
const pageLoadTime = /* time taken by a webpage to load */;
```

The value of `pageLoadTime` is not known before the page load, so it's named normally. But it's still a constant because it doesn't change after the assignment.

In other words, capital-named constants are only used as aliases for "hard-coded" values.

Name things right

Talking about variables, there's one more extremely important thing.

A variable name should have a clean, obvious meaning, describing the data that it stores.

Variable naming is one of the most important and complex skills in programming. A glance at variable names can reveal which code was written by a beginner versus an experienced developer.

In a real project, most of the time is spent modifying and extending an existing code base rather than writing something completely separate from scratch. When we return to some code after doing something else for a while, it's much easier to find information that is well-labelled. Or, in other words, when the variables have good names.

Please spend time thinking about the right name for a variable before declaring it. Doing so will repay you handsomely.

Some good-to-follow rules are:

- Use human-readable names like `userName` or `shoppingCart`.
- Stay away from abbreviations or short names like `a`, `b`, and `c`, unless you know what you're doing.
- Make names maximally descriptive and concise. Examples of bad names are `data` and `value`. Such names say nothing. It's only okay to use them if the context of the code makes it exceptionally obvious which data or value the variable is referencing.
- Agree on terms within your team and in your mind. If a site visitor is called a "user" then we should name related variables `currentUser` or `newUser` instead of `currentVisitor` or `newManInTown`.

Sounds simple? Indeed it is, but creating descriptive and concise variable names in practice is not. Go for it.

Reuse or create?

And the last note. There are some lazy programmers who, instead of declaring new variables, tend to reuse existing ones.

As a result, their variables are like boxes into which people throw different things without changing their stickers. What's inside the box now? Who knows? We need to come closer and check.

Such programmers save a little bit on variable declaration but lose ten times more on debugging.

An extra variable is good, not evil.

Modern JavaScript minifiers and browsers optimize code well enough, so it won't create performance issues. Using different variables for different values can even help the engine optimize your code.

Summary

We can declare variables to store data by using the `var`, `let`, or `const` keywords.

- `let` – is a modern variable declaration.
- `var` – is an old-school variable declaration. Normally we don't use it at all, but we'll cover subtle differences from `let` in the chapter [The old "var"](#), just in case you need them.
- `const` – is like `let`, but the value of the variable can't be changed.

Variables should be named in a way that allows us to easily understand what's inside them.

Tasks

Working with variables

importance: 2

1. Declare two variables: `admin` and `name`.
2. Assign the value `"John"` to `name`.
3. Copy the value from `name` to `admin`.
4. Show the value of `admin` using `alert` (must output "John").

[To solution](#)

Giving the right name

importance: 3

1. Create a variable with the name of our planet. How would you name such a variable?
2. Create a variable to store the name of a current visitor to a website. How would you name that variable?

[To solution](#)

Uppercase const?

importance: 4

Examine the following code:

```
const birthday = '18.04.1982';  
const age = someCode(birthday);
```

Here we have a constant `birthday` for the date, and also the `age` constant.

The `age` is calculated from `birthday` using `someCode()`, which means a function call that we didn't explain yet (we will soon!), but the details don't matter here, the point is that `age` is calculated somehow based on the `birthday`.

Would it be right to use upper case for `birthday`? For `age`? Or even for both?

```
const BIRTHDAY = '18.04.1982'; // make birthday uppercase?

const AGE = someCode(BIRTHDAY); // make age uppercase?
```

[To solution](#)

Data types

A value in JavaScript is always of a certain type. For example, a string or a number.

There are eight basic data types in JavaScript. Here, we'll cover them in general and in the next chapters we'll talk about each of them in detail.

We can put any type in a variable. For example, a variable can at one moment be a string and then store a number:

```
// no error
let message = "hello";
message = 123456;
```

Programming languages that allow such things, such as JavaScript, are called “dynamically typed”, meaning that there exist data types, but variables are not bound to any of them.

Number

```
let n = 123;
n = 12.345;
```

The *number* type represents both integer and floating point numbers.

There are many operations for numbers, e.g. multiplication `*`, division `/`, addition `+`, subtraction `-`, and so on.

Besides regular numbers, there are so-called “special numeric values” which also belong to this data type: `Infinity`, `-Infinity` and `NaN`.

- `Infinity` represents the mathematical [Infinity](#) ∞ . It is a special value that's greater than any number.

We can get it as a result of division by zero:

```
alert( 1 / 0 ); // Infinity
```

Or just reference it directly:

```
alert( Infinity ); // Infinity
```

- `NaN` represents a computational error. It is a result of an incorrect or an undefined mathematical operation, for instance:

```
alert( "not a number" / 2 ); // NaN, such division is erroneous
```

`NaN` is sticky. Any further mathematical operation on `NaN` returns `NaN`:

```
alert( NaN + 1 ); // NaN
alert( 3 * NaN ); // NaN
alert( "not a number" / 2 - 1 ); // NaN
```

So, if there's a `NaN` somewhere in a mathematical expression, it propagates to the whole result (there's only one exception to that: `NaN ** 0` is `1`).

Mathematical operations are safe

Doing maths is “safe” in JavaScript. We can do anything: divide by zero, treat non-numeric strings as numbers, etc.

The script will never stop with a fatal error (“die”). At worst, we’ll get `NaN` as the result.

Special numeric values formally belong to the “number” type. Of course they are not numbers in the common sense of this word.

We’ll see more about working with numbers in the chapter [Numbers](#).

BigInt

In JavaScript, the “number” type cannot safely represent integer values larger than $(2^{53}-1)$ (that’s `9007199254740991`), or less than $-(2^{53}-1)$ for negatives.

To be really precise, the “number” type can store larger integers (up to $1.7976931348623157 \cdot 10^{308}$), but outside of the safe integer range $\pm(2^{53}-1)$ there’ll be a precision error, because not all digits fit into the fixed 64-bit storage. So an “approximate” value may be stored.

For example, these two numbers (right above the safe range) are the same:

```
console.log(9007199254740991 + 1); // 9007199254740992
console.log(9007199254740991 + 2); // 9007199254740992
```

So to say, all odd integers greater than $(2^{53}-1)$ can’t be stored at all in the “number” type.

For most purposes $\pm(2^{53}-1)$ range is quite enough, but sometimes we need the entire range of really big integers, e.g. for cryptography or microsecond-precision timestamps.

`BigInt` type was recently added to the language to represent integers of arbitrary length.

A `BigInt` value is created by appending `n` to the end of an integer:

```
// the "n" at the end means it's a BigInt
const bigInt = 1234567890123456789012345678901234567890n;
```

As `BigInt` numbers are rarely needed, we don’t cover them here, but devoted them a separate chapter [BigInt](#). Read it when you need such big numbers.

String

A string in JavaScript must be surrounded by quotes.

```
let str = "Hello";
let str2 = 'Single quotes are ok too';
let phrase = `can embed another ${str}`;
```

In JavaScript, there are 3 types of quotes.

1. Double quotes: `"Hello"`.
2. Single quotes: `'Hello'`.
3. Backticks: ``Hello``.

Double and single quotes are “simple” quotes. There’s practically no difference between them in JavaScript.

Backticks are “extended functionality” quotes. They allow us to embed variables and expressions into a string by wrapping them in `${...}`, for example:

```
let name = "John";

// embed a variable
alert(`Hello, ${name}!`); // Hello, John!
```

```
// embed an expression
alert( `the result is ${1 + 2}` ); // the result is 3
```

The expression inside `${...}` is evaluated and the result becomes a part of the string. We can put anything in there: a variable like `name` or an arithmetical expression like `1 + 2` or something more complex.

Please note that this can only be done in backticks. Other quotes don't have this embedding functionality!

```
alert( "the result is ${1 + 2}" ); // the result is ${1 + 2} (double quotes do nothing)
```

We'll cover strings more thoroughly in the chapter [Strings](#).

There is no *character* type.

In some languages, there is a special “character” type for a single character. For example, in the C language and in Java it is called “char”.

In JavaScript, there is no such type. There's only one type: `string`. A string may consist of zero characters (be empty), one character or many of them.

Boolean (logical type)

The boolean type has only two values: `true` and `false`.

This type is commonly used to store yes/no values: `true` means “yes, correct”, and `false` means “no, incorrect”.

For instance:

```
let nameFieldChecked = true; // yes, name field is checked
let ageFieldChecked = false; // no, age field is not checked
```

Boolean values also come as a result of comparisons:

```
let isGreater = 4 > 1;

alert( isGreater ); // true (the comparison result is "yes")
```

We'll cover booleans more deeply in the chapter [Logical operators](#).

The “null” value

The special `null` value does not belong to any of the types described above.

It forms a separate type of its own which contains only the `null` value:

```
let age = null;
```

In JavaScript, `null` is not a “reference to a non-existing object” or a “null pointer” like in some other languages.

It's just a special value which represents “nothing”, “empty” or “value unknown”.

The code above states that `age` is unknown.

The “undefined” value

The special value `undefined` also stands apart. It makes a type of its own, just like `null`.

The meaning of `undefined` is “value is not assigned”.

If a variable is declared, but not assigned, then its value is `undefined`:

```
let age;

alert(age); // shows "undefined"
```

Technically, it is possible to explicitly assign `undefined` to a variable:

```
let age = 100;

// change the value to undefined
age = undefined;

alert(age); // "undefined"
```

...But we don't recommend doing that. Normally, one uses `null` to assign an "empty" or "unknown" value to a variable, while `undefined` is reserved as a default initial value for unassigned things.

Objects and Symbols

The `object` type is special.

All other types are called "primitive" because their values can contain only a single thing (be it a string or a number or whatever). In contrast, objects are used to store collections of data and more complex entities.

Being that important, objects deserve a special treatment. We'll deal with them later in the chapter [Objects](#), after we learn more about primitives.

The `symbol` type is used to create unique identifiers for objects. We have to mention it here for the sake of completeness, but also postpone the details till we know objects.

The `typeof` operator

The `typeof` operator returns the type of the operand. It's useful when we want to process values of different types differently or just want to do a quick check.

A call to `typeof x` returns a string with the type name:

```
typeof undefined // "undefined"

typeof 0 // "number"

typeof 10n // "bigint"

typeof true // "boolean"

typeof "foo" // "string"

typeof Symbol("id") // "symbol"

typeof Math // "object" (1)

typeof null // "object" (2)

typeof alert // "function" (3)
```

The last three lines may need additional explanation:

1. `Math` is a built-in object that provides mathematical operations. We will learn it in the chapter [Numbers](#). Here, it serves just as an example of an object.
2. The result of `typeof null` is `"object"`. That's an officially recognized error in `typeof`, coming from very early days of JavaScript and kept for compatibility. Definitely, `null` is not an object. It is a special value with a separate type of its own. The behavior of `typeof` is wrong here.
3. The result of `typeof alert` is `"function"`, because `alert` is a function. We'll study functions in the next chapters where we'll also see that there's no special "function" type in JavaScript. Functions belong to the object type. But `typeof` treats them differently, returning `"function"`. That also comes from the early days of JavaScript. Technically, such behavior isn't correct, but can be convenient in practice.

The `typeof(x)` syntax

You may also come across another syntax: `typeof(x)`. It's the same as `typeof x`.

To put it clear: `typeof` is an operator, not a function. The parentheses here aren't a part of `typeof`. It's the kind of parentheses used for mathematical grouping.

Usually, such parentheses contain a mathematical expression, such as `(2 + 2)`, but here they contain only one argument `(x)`. Syntactically, they allow to avoid a space between the `typeof` operator and its argument, and some people like it.

Some people prefer `typeof(x)`, although the `typeof x` syntax is much more common.

Summary

There are 8 basic data types in JavaScript.

- Seven primitive data types:
 - `number` for numbers of any kind: integer or floating-point, integers are limited by $\pm(2^{53}-1)$.
 - `bigint` for integer numbers of arbitrary length.
 - `string` for strings. A string may have zero or more characters, there's no separate single-character type.
 - `boolean` for `true/false`.
 - `null` for unknown values – a standalone type that has a single value `null`.
 - `undefined` for unassigned values – a standalone type that has a single value `undefined`.
 - `symbol` for unique identifiers.
- And one non-primitive data type:
 - `object` for more complex data structures.

The `typeof` operator allows us to see which type is stored in a variable.

- Usually used as `typeof x`, but `typeof(x)` is also possible.
- Returns a string with the name of the type, like `"string"`.
- For `null` returns `"object"` – this is an error in the language, it's not actually an object.

In the next chapters, we'll concentrate on primitive values and once we're familiar with them, we'll move on to objects.

Tasks

String quotes

importance: 5

What is the output of the script?

```
let name = "Ilya";

alert( `hello ${1}` ); // ?

alert( `hello ${"name"}` ); // ?

alert( `hello ${name}` ); // ?
```

[To solution](#)

Interaction: alert, prompt, confirm

As we'll be using the browser as our demo environment, let's see a couple of functions to interact with the user: `alert`, `prompt` and `confirm`.

alert

This one we've seen already. It shows a message and waits for the user to press "OK".

For example:

```
alert("Hello");
```

The mini-window with the message is called a *modal window*. The word “modal” means that the visitor can't interact with the rest of the page, press other buttons, etc, until they have dealt with the window. In this case – until they press “OK”.

prompt

The function `prompt` accepts two arguments:

```
result = prompt(title, [default]);
```

It shows a modal window with a text message, an input field for the visitor, and the buttons OK/Cancel.

title

The text to show the visitor.

default

An optional second parameter, the initial value for the input field.

The square brackets in syntax [...]

The square brackets around `default` in the syntax above denote that the parameter is optional, not required.

The visitor can type something in the prompt input field and press OK. Then we get that text in the `result`. Or they can cancel the input by pressing Cancel or hitting the `Esc` key, then we get `null` as the `result`.

The call to `prompt` returns the text from the input field or `null` if the input was canceled.

For instance:

```
let age = prompt('How old are you?', 100);  
alert(`You are ${age} years old!`); // You are 100 years old!
```

In IE: always supply a default

The second parameter is optional, but if we don't supply it, Internet Explorer will insert the text `"undefined"` into the prompt.

Run this code in Internet Explorer to see:

```
let test = prompt("Test");
```

So, for prompts to look good in IE, we recommend always providing the second argument:

```
let test = prompt("Test", ''); // <-- for IE
```

confirm

The syntax:

```
result = confirm(question);
```

The function `confirm` shows a modal window with a `question` and two buttons: OK and Cancel.

The result is `true` if OK is pressed and `false` otherwise.

For example:

```
let isBoss = confirm("Are you the boss?");  
  
alert( isBoss ); // true if OK is pressed
```

Summary

We covered 3 browser-specific functions to interact with visitors:

`alert`

shows a message.

`prompt`

shows a message asking the user to input text. It returns the text or, if Cancel button or `Esc` is clicked, `null`.

`confirm`

shows a message and waits for the user to press “OK” or “Cancel”. It returns `true` for OK and `false` for Cancel/`Esc`.

All these methods are modal: they pause script execution and don't allow the visitor to interact with the rest of the page until the window has been dismissed.

There are two limitations shared by all the methods above:

1. The exact location of the modal window is determined by the browser. Usually, it's in the center.
2. The exact look of the window also depends on the browser. We can't modify it.

That is the price for simplicity. There are other ways to show nicer windows and richer interaction with the visitor, but if “bells and whistles” do not matter much, these methods work just fine.

✔ Tasks

A simple page

importance: 4

Create a web-page that asks for a name and outputs it.

[Run the demo](#)

[To solution](#)

Type Conversions

Most of the time, operators and functions automatically convert the values given to them to the right type.

For example, `alert` automatically converts any value to a string to show it. Mathematical operations convert values to numbers.

There are also cases when we need to explicitly convert a value to the expected type.

Not talking about objects yet

In this chapter, we won't cover objects. For now, we'll just be talking about primitives.

Later, after we learn about objects, in the chapter [Object to primitive conversion](#) we'll see how objects fit in.

String Conversion

String conversion happens when we need the string form of a value.

For example, `alert(value)` does it to show the value.

We can also call the `String(value)` function to convert a value to a string:

```
let value = true;
alert(typeof value); // boolean

value = String(value); // now value is a string "true"
alert(typeof value); // string
```

String conversion is mostly obvious. A `false` becomes `"false"`, `null` becomes `"null"`, etc.

Numeric Conversion

Numeric conversion in mathematical functions and expressions happens automatically.

For example, when division `/` is applied to non-numbers:

```
alert( "6" / "2" ); // 3, strings are converted to numbers
```

We can use the `Number(value)` function to explicitly convert a `value` to a number:

```
let str = "123";
alert(typeof str); // string

let num = Number(str); // becomes a number 123

alert(typeof num); // number
```

Explicit conversion is usually required when we read a value from a string-based source like a text form but expect a number to be entered.

If the string is not a valid number, the result of such a conversion is `NaN`. For instance:

```
let age = Number("an arbitrary string instead of a number");

alert(age); // NaN, conversion failed
```

Numeric conversion rules:

Value	Becomes...
<code>undefined</code>	<code>NaN</code>
<code>null</code>	<code>0</code>
<code>true</code> and <code>false</code>	<code>1</code> and <code>0</code>
<code>string</code>	Whitespaces (includes spaces, tabs <code>\t</code> , newlines <code>\n</code> etc.) from the start and end are removed. If the remaining string is empty, the result is <code>0</code> . Otherwise, the number is "read" from the string. An error gives <code>NaN</code> .

Examples:

```
alert( Number(" 123 ") ); // 123
alert( Number("123z") ); // NaN (error reading a number at "z")
alert( Number(true) ); // 1
alert( Number(false) ); // 0
```

Please note that `null` and `undefined` behave differently here: `null` becomes zero while `undefined` becomes `NaN`.

Most mathematical operators also perform such conversion, we'll see that in the next chapter.

Boolean Conversion

Boolean conversion is the simplest one.

It happens in logical operations (later we'll meet condition tests and other similar things) but can also be performed explicitly with a call to `Boolean(value)`.

The conversion rule:

- Values that are intuitively “empty”, like `0`, an empty string, `null`, `undefined`, and `NaN`, become `false`.
- Other values become `true`.

For instance:

```
alert( Boolean(1) ); // true
alert( Boolean(0) ); // false

alert( Boolean("hello") ); // true
alert( Boolean("") ); // false
```

 **Please note: the string with zero `"0"` is `true`**

Some languages (namely PHP) treat `"0"` as `false`. But in JavaScript, a non-empty string is always `true`.

```
alert( Boolean("0") ); // true
alert( Boolean(" ") ); // spaces, also true (any non-empty string is true)
```

Summary

The three most widely used type conversions are to string, to number, and to boolean.

String Conversion – Occurs when we output something. Can be performed with `String(value)`. The conversion to string is usually obvious for primitive values.

Numeric Conversion – Occurs in math operations. Can be performed with `Number(value)`.

The conversion follows the rules:

Value	Becomes...
<code>undefined</code>	<code>NaN</code>
<code>null</code>	<code>0</code>
<code>true</code> / <code>false</code>	<code>1</code> / <code>0</code>
<code>string</code>	The string is read “as is”, whitespaces (includes spaces, tabs <code>\t</code> , newlines <code>\n</code> etc.) from both sides are ignored. An empty string becomes <code>0</code> . An error gives <code>NaN</code> .

Boolean Conversion – Occurs in logical operations. Can be performed with `Boolean(value)`.

Follows the rules:

Value	Becomes...
<code>0</code> , <code>null</code> , <code>undefined</code> , <code>NaN</code> , <code>""</code>	<code>false</code>
any other value	<code>true</code>

Most of these rules are easy to understand and memorize. The notable exceptions where people usually make mistakes are:

- `undefined` is `NaN` as a number, not `0`.
- `"0"` and space-only strings like `" "` are `true` as a boolean.

Objects aren’t covered here. We’ll return to them later in the chapter [Object to primitive conversion](#) that is devoted exclusively to objects after we learn more basic things about JavaScript.

Basic operators, maths

We know many operators from school. They are things like addition `+`, multiplication `*`, subtraction `-`, and so on.

In this chapter, we’ll start with simple operators, then concentrate on JavaScript-specific aspects, not covered by school arithmetic.

Terms: “unary”, “binary”, “operand”

Before we move on, let's grasp some common terminology.

- An *operand* – is what operators are applied to. For instance, in the multiplication of `5 * 2` there are two operands: the left operand is `5` and the right operand is `2`. Sometimes, people call these “arguments” instead of “operands”.
- An operator is *unary* if it has a single operand. For example, the unary negation `-` reverses the sign of a number:

```
let x = 1;
x = -x;
alert( x ); // -1, unary negation was applied
```

- An operator is *binary* if it has two operands. The same minus exists in binary form as well:

```
let x = 1, y = 3;
alert( y - x ); // 2, binary minus subtracts values
```

Formally, in the examples above we have two different operators that share the same symbol: the negation operator, a unary operator that reverses the sign, and the subtraction operator, a binary operator that subtracts one number from another.

Maths

The following math operations are supported:

- Addition `+`,
- Subtraction `-`,
- Multiplication `*`,
- Division `/`,
- Remainder `%`,
- Exponentiation `**`.

The first four are straightforward, while `%` and `**` need a few words about them.

Remainder %

The remainder operator `%`, despite its appearance, is not related to percents.

The result of `a % b` is the [remainder](#) of the integer division of `a` by `b`.

For instance:

```
alert( 5 % 2 ); // 1, the remainder of 5 divided by 2
alert( 8 % 3 ); // 2, the remainder of 8 divided by 3
alert( 8 % 4 ); // 0, the remainder of 8 divided by 4
```

Exponentiation **

The exponentiation operator `a ** b` raises `a` to the power of `b`.

In school maths, we write that as a^b .

For instance:

```
alert( 2 ** 2 ); // 22 = 4
alert( 2 ** 3 ); // 23 = 8
alert( 2 ** 4 ); // 24 = 16
```

Just like in maths, the exponentiation operator is defined for non-integer numbers as well.

For example, a square root is an exponentiation by $\frac{1}{2}$:

```
alert( 4 ** (1/2) ); // 2 (power of 1/2 is the same as a square root)
alert( 8 ** (1/3) ); // 2 (power of 1/3 is the same as a cubic root)
```

String concatenation with binary +

Let's meet the features of JavaScript operators that are beyond school arithmetics.

Usually, the plus operator `+` sums numbers.

But, if the binary `+` is applied to strings, it merges (concatenates) them:

```
let s = "my" + "string";
alert(s); // mystring
```

Note that if any of the operands is a string, then the other one is converted to a string too.

For example:

```
alert( '1' + 2 ); // "12"
alert( 2 + '1' ); // "21"
```

See, it doesn't matter whether the first operand is a string or the second one.

Here's a more complex example:

```
alert(2 + 2 + '1' ); // "41" and not "221"
```

Here, operators work one after another. The first `+` sums two numbers, so it returns `4`, then the next `+` adds the string `1` to it, so it's like `4 + '1' = '41'`.

```
alert('1' + 2 + 2); // "122" and not "14"
```

Here, the first operand is a string, the compiler treats the other two operands as strings too. The `2` gets concatenated to `'1'`, so it's like `'1' + 2 = "12"` and `"12" + 2 = "122"`.

The binary `+` is the only operator that supports strings in such a way. Other arithmetic operators work only with numbers and always convert their operands to numbers.

Here's the demo for subtraction and division:

```
alert( 6 - '2' ); // 4, converts '2' to a number
alert( '6' / '2' ); // 3, converts both operands to numbers
```

Numeric conversion, unary +

The plus `+` exists in two forms: the binary form that we used above and the unary form.

The unary plus or, in other words, the plus operator `+` applied to a single value, doesn't do anything to numbers. But if the operand is not a number, the unary plus converts it into a number.

For example:

```
// No effect on numbers
let x = 1;
alert( +x ); // 1

let y = -2;
alert( +y ); // -2

// Converts non-numbers
alert( +true ); // 1
alert( +"" ); // 0
```

It actually does the same thing as `Number(...)`, but is shorter.

The need to convert strings to numbers arises very often. For example, if we are getting values from HTML form fields, they are usually strings. What if we want to sum them?

The binary plus would add them as strings:

```
let apples = "2";
let oranges = "3";

alert( apples + oranges ); // "23", the binary plus concatenates strings
```

If we want to treat them as numbers, we need to convert and then sum them:

```
let apples = "2";
let oranges = "3";

// both values converted to numbers before the binary plus
alert( +apples + +oranges ); // 5

// the longer variant
// alert( Number(apples) + Number(oranges) ); // 5
```

From a mathematician's standpoint, the abundance of pluses may seem strange. But from a programmer's standpoint, there's nothing special: unary pluses are applied first, they convert strings to numbers, and then the binary plus sums them up.

Why are unary pluses applied to values before the binary ones? As we're going to see, that's because of their *higher precedence*.

Operator precedence

If an expression has more than one operator, the execution order is defined by their *precedence*, or, in other words, the default priority order of operators.

From school, we all know that the multiplication in the expression `1 + 2 * 2` should be calculated before the addition. That's exactly the precedence thing. The multiplication is said to have a *higher precedence* than the addition.

Parentheses override any precedence, so if we're not satisfied with the default order, we can use them to change it. For example, write `(1 + 2) * 2`.

There are many operators in JavaScript. Every operator has a corresponding precedence number. The one with the larger number executes first. If the precedence is the same, the execution order is from left to right.

Here's an extract from the [precedence table](#) (you don't need to remember this, but note that unary operators are higher than corresponding binary ones):

Precedence	Name	Sign
...	...	...
14	unary plus	+
14	unary negation	-
13	exponentiation	**
12	multiplication	*
12	division	/
11	addition	+
11	subtraction	-
...	...	...
2	assignment	=
...	...	...

As we can see, the "unary plus" has a priority of `14` which is higher than the `11` of "addition" (binary plus). That's why, in the expression `" +apples + +oranges "`, unary pluses work before the addition.

Assignment

Let's note that an assignment `=` is also an operator. It is listed in the precedence table with the very low priority of `2`.

That's why, when we assign a variable, like `x = 2 * 2 + 1`, the calculations are done first and then the `=` is evaluated, storing the result in `x`.

```
let x = 2 * 2 + 1;
alert( x ); // 5
```

Assignment = returns a value

The fact of `=` being an operator, not a "magical" language construct has an interesting implication.

All operators in JavaScript return a value. That's obvious for `+` and `-`, but also true for `=`.

The call `x = value` writes the `value` into `x` and then returns it.

Here's a demo that uses an assignment as part of a more complex expression:

```
let a = 1;
let b = 2;

let c = 3 - (a = b + 1);

alert( a ); // 3
alert( c ); // 0
```

In the example above, the result of expression `(a = b + 1)` is the value which was assigned to `a` (that is `3`). It is then used for further evaluations.

Funny code, isn't it? We should understand how it works, because sometimes we see it in JavaScript libraries.

Although, please don't write the code like that. Such tricks definitely don't make code clearer or readable.

Chaining assignments

Another interesting feature is the ability to chain assignments:

```
let a, b, c;

a = b = c = 2 + 2;

alert( a ); // 4
alert( b ); // 4
alert( c ); // 4
```

Chained assignments evaluate from right to left. First, the rightmost expression `2 + 2` is evaluated and then assigned to the variables on the left: `c`, `b` and `a`. At the end, all the variables share a single value.

Once again, for the purposes of readability it's better to split such code into few lines:

```
c = 2 + 2;
b = c;
a = c;
```

That's easier to read, especially when eye-scanning the code fast.

Modify-in-place

We often need to apply an operator to a variable and store the new result in that same variable.

For example:

```
let n = 2;
n = n + 5;
n = n * 2;
```

This notation can be shortened using the operators `+=` and `*=`:

```
let n = 2;
n += 5; // now n = 7 (same as n = n + 5)
n *= 2; // now n = 14 (same as n = n * 2)

alert( n ); // 14
```

Short “modify-and-assign” operators exist for all arithmetical and bitwise operators: `/=`, `-=`, etc.

Such operators have the same precedence as a normal assignment, so they run after most other calculations:

```
let n = 2;

n *= 3 + 5; // right part evaluated first, same as n *= 8

alert( n ); // 16
```

Increment/decrement

Increasing or decreasing a number by one is among the most common numerical operations.

So, there are special operators for it:

- **Increment** `++` increases a variable by 1:

```
let counter = 2;
counter++; // works the same as counter = counter + 1, but is shorter
alert( counter ); // 3
```

- **Decrement** `--` decreases a variable by 1:

```
let counter = 2;
counter--; // works the same as counter = counter - 1, but is shorter
alert( counter ); // 1
```

Important:

Increment/decrement can only be applied to variables. Trying to use it on a value like `5++` will give an error.

The operators `++` and `--` can be placed either before or after a variable.

- When the operator goes after the variable, it is in “postfix form”: `counter++`.
- The “prefix form” is when the operator goes before the variable: `++counter`.

Both of these statements do the same thing: increase `counter` by 1.

Is there any difference? Yes, but we can only see it if we use the returned value of `++/--`.

Let's clarify. As we know, all operators return a value. Increment/decrement is no exception. The prefix form returns the new value while the postfix form returns the old value (prior to increment/decrement).

To see the difference, here's an example:

```
let counter = 1;
let a = ++counter; // (*)

alert(a); // 2
```

In the line `(*)`, the *prefix* form `++counter` increments `counter` and returns the new value, 2. So, the `alert` shows 2.

Now, let's use the postfix form:

```
let counter = 1;
let a = counter++; // (*) changed ++counter to counter++

alert(a); // 1
```

In the line `(*)`, the *postfix* form `counter++` also increments `counter` but returns the *old* value (prior to increment). So, the `alert` shows `1`.

To summarize:

- If the result of increment/decrement is not used, there is no difference in which form to use:

```
let counter = 0;
counter++;
++counter;
alert( counter ); // 2, the lines above did the same
```

- If we'd like to increase a value *and* immediately use the result of the operator, we need the prefix form:

```
let counter = 0;
alert( ++counter ); // 1
```

- If we'd like to increment a value but use its previous value, we need the postfix form:

```
let counter = 0;
alert( counter++ ); // 0
```

Increment/decrement among other operators

The operators `++/-` can be used inside expressions as well. Their precedence is higher than most other arithmetical operations.

For instance:

```
let counter = 1;
alert( 2 * ++counter ); // 4
```

Compare with:

```
let counter = 1;
alert( 2 * counter++ ); // 2, because counter++ returns the "old" value
```

Though technically okay, such notation usually makes code less readable. One line does multiple things – not good.

While reading code, a fast “vertical” eye-scan can easily miss something like `counter++` and it won't be obvious that the variable increased.

We advise a style of “one line – one action”:

```
let counter = 1;
alert( 2 * counter );
counter++;
```

Bitwise operators

Bitwise operators treat arguments as 32-bit integer numbers and work on the level of their binary representation.

These operators are not JavaScript-specific. They are supported in most programming languages.

The list of operators:

- AND (`&`)
- OR (`|`)
- XOR (`^`)
- NOT (`~`)
- LEFT SHIFT (`<<`)
- RIGHT SHIFT (`>>`)
- ZERO-FILL RIGHT SHIFT (`>>>`)

These operators are used very rarely, when we need to fiddle with numbers on the very lowest (bitwise) level. We won't need these operators any time soon, as web development has little use of them, but in some special areas, such as cryptography, they are useful. You can read the [Bitwise Operators](#) chapter on MDN when a need arises.

Comma

The comma operator `,` is one of the rarest and most unusual operators. Sometimes, it's used to write shorter code, so we need to know it in order to understand what's going on.

The comma operator allows us to evaluate several expressions, dividing them with a comma `,`. Each of them is evaluated but only the result of the last one is returned.

For example:

```
let a = (1 + 2, 3 + 4);
alert( a ); // 7 (the result of 3 + 4)
```

Here, the first expression `1 + 2` is evaluated and its result is thrown away. Then, `3 + 4` is evaluated and returned as the result.

Comma has a very low precedence

Please note that the comma operator has very low precedence, lower than `=`, so parentheses are important in the example above.

Without them: `a = 1 + 2, 3 + 4` evaluates `+` first, summing the numbers into `a = 3, 7`, then the assignment operator `=` assigns `a = 3`, and the rest is ignored. It's like `(a = 1 + 2), 3 + 4`.

Why do we need an operator that throws away everything except the last expression?

Sometimes, people use it in more complex constructs to put several actions in one line.

For example:

```
// three operations in one line
for (a = 1, b = 3, c = a * b; a < 10; a++) {
  ...
}
```

Such tricks are used in many JavaScript frameworks. That's why we're mentioning them. But usually they don't improve code readability so we should think well before using them.

Tasks

The postfix and prefix forms

importance: 5

What are the final values of all variables `a`, `b`, `c` and `d` after the code below?

```
let a = 1, b = 1;

let c = ++a; // ?
let d = b++; // ?
```

[To solution](#)

Assignment result

importance: 3

What are the values of `a` and `x` after the code below?

```
let a = 2;

let x = 1 + (a *= 2);
```

[To solution](#)

Type conversions

importance: 5

What are results of these expressions?

```
" " + 1 + 0
" " - 1 + 0
true + false
6 / "3"
"2" * "3"
4 + 5 + "px"
"$" + 4 + 5
"4" - 2
"4px" - 2
"  -9  " + 5
"  -9  " - 5
null + 1
undefined + 1
" \t \n" - 2
```

Think well, write down and then compare with the answer.

[To solution](#)

Fix the addition

importance: 5

Here's a code that asks the user for two numbers and shows their sum.

It works incorrectly. The output in the example below is `12` (for default prompt values).

Why? Fix it. The result should be `3`.

```
let a = prompt("First number?", 1);
let b = prompt("Second number?", 2);

alert(a + b); // 12
```

[To solution](#)

Comparisons

We know many comparison operators from maths.

In JavaScript they are written like this:

- Greater/less than: `a > b`, `a < b`.
- Greater/less than or equals: `a >= b`, `a <= b`.
- Equals: `a == b`, please note the double equality sign `==` means the equality test, while a single one `a = b` means an assignment.

- Not equals: In maths the notation is $\neq$, but in JavaScript it's written as `a !== b`.

In this article we'll learn more about different types of comparisons, how JavaScript makes them, including important peculiarities.

At the end you'll find a good recipe to avoid "JavaScript quirks"-related issues.

Boolean is the result

All comparison operators return a boolean value:

- `true` – means "yes", "correct" or "the truth".
- `false` – means "no", "wrong" or "not the truth".

For example:

```
alert( 2 > 1 ); // true (correct)
alert( 2 == 1 ); // false (wrong)
alert( 2 !== 1 ); // true (correct)
```

A comparison result can be assigned to a variable, just like any value:

```
let result = 5 > 4; // assign the result of the comparison
alert( result ); // true
```

String comparison

To see whether a string is greater than another, JavaScript uses the so-called "dictionary" or "lexicographical" order.

In other words, strings are compared letter-by-letter.

For example:

```
alert( 'Z' > 'A' ); // true
alert( 'Glow' > 'Glee' ); // true
alert( 'Bee' > 'Be' ); // true
```

The algorithm to compare two strings is simple:

1. Compare the first character of both strings.
2. If the first character from the first string is greater (or less) than the other string's, then the first string is greater (or less) than the second. We're done.
3. Otherwise, if both strings' first characters are the same, compare the second characters the same way.
4. Repeat until the end of either string.
5. If both strings end at the same length, then they are equal. Otherwise, the longer string is greater.

In the first example above, the comparison `'Z' > 'A'` gets to a result at the first step.

The second comparison `'Glow' > 'Glee'` needs more steps as strings are compared character-by-character:

1. `G` is the same as `G`.
2. `l` is the same as `l`.
3. `o` is greater than `e`. Stop here. The first string is greater.

Not a real dictionary, but Unicode order

The comparison algorithm given above is roughly equivalent to the one used in dictionaries or phone books, but it's not exactly the same.

For instance, case matters. A capital letter `"A"` is not equal to the lowercase `"a"`. Which one is greater? The lowercase `"a"`. Why? Because the lowercase character has a greater index in the internal encoding table JavaScript uses (Unicode). We'll get back to specific details and consequences of this in the chapter [Strings](#).

Comparison of different types

When comparing values of different types, JavaScript converts the values to numbers.

For example:

```
alert( '2' > 1 ); // true, string '2' becomes a number 2
alert( '01' == 1 ); // true, string '01' becomes a number 1
```

For boolean values, `true` becomes `1` and `false` becomes `0`.

For example:

```
alert( true == 1 ); // true
alert( false == 0 ); // true
```

A funny consequence

It is possible that at the same time:

- Two values are equal.
- One of them is `true` as a boolean and the other one is `false` as a boolean.

For example:

```
let a = 0;
alert( Boolean(a) ); // false

let b = "0";
alert( Boolean(b) ); // true

alert(a == b); // true!
```

From JavaScript's standpoint, this result is quite normal. An equality check converts values using the numeric conversion (hence `"0"` becomes `0`), while the explicit `Boolean` conversion uses another set of rules.

Strict equality

A regular equality check `==` has a problem. It cannot differentiate `0` from `false`:

```
alert( 0 == false ); // true
```

The same thing happens with an empty string:

```
alert( '' == false ); // true
```

This happens because operands of different types are converted to numbers by the equality operator `==`. An empty string, just like `false`, becomes a zero.

What to do if we'd like to differentiate `0` from `false`?

A strict equality operator `===` checks the equality without type conversion.

In other words, if `a` and `b` are of different types, then `a === b` immediately returns `false` without an attempt to convert them.

Let's try it:

```
alert( 0 === false ); // false, because the types are different
```

There is also a "strict non-equality" operator `!==` analogous to `!=`.

The strict equality operator is a bit longer to write, but makes it obvious what's going on and leaves less room for errors.

Comparison with null and undefined

There's a non-intuitive behavior when `null` or `undefined` are compared to other values.

For a strict equality check `===`

These values are different, because each of them is a different type.

```
alert( null === undefined ); // false
```

For a non-strict check `==`

There's a special rule. These two are a "sweet couple": they equal each other (in the sense of `==`), but not any other value.

```
alert( null == undefined ); // true
```

For maths and other comparisons `<` `>` `<=` `>=`

`null/undefined` are converted to numbers: `null` becomes `0`, while `undefined` becomes `NaN`.

Now let's see some funny things that happen when we apply these rules. And, what's more important, how to not fall into a trap with them.

Strange result: null vs 0

Let's compare `null` with a zero:

```
alert( null > 0 ); // (1) false
alert( null == 0 ); // (2) false
alert( null >= 0 ); // (3) true
```

Mathematically, that's strange. The last result states that "`null` is greater than or equal to zero", so in one of the comparisons above it must be `true`, but they are both false.

The reason is that an equality check `==` and comparisons `>` `<` `>=` `<=` work differently. Comparisons convert `null` to a number, treating it as `0`. That's why (3) `null >= 0` is true and (1) `null > 0` is false.

On the other hand, the equality check `==` for `undefined` and `null` is defined such that, without any conversions, they equal each other and don't equal anything else. That's why (2) `null == 0` is false.

An incomparable undefined

The value `undefined` shouldn't be compared to other values:

```
alert( undefined > 0 ); // false (1)
alert( undefined < 0 ); // false (2)
alert( undefined == 0 ); // false (3)
```

Why does it dislike zero so much? Always false!

We get these results because:

- Comparisons (1) and (2) return `false` because `undefined` gets converted to `NaN` and `NaN` is a special numeric value which returns `false` for all comparisons.
- The equality check (3) returns `false` because `undefined` only equals `null`, `undefined`, and no other value.

Avoid problems

Why did we go over these examples? Should we remember these peculiarities all the time? Well, not really. Actually, these tricky things will gradually become familiar over time, but there's a solid way to avoid problems with them:

- Treat any comparison with `undefined/null` except the strict equality `===` with exceptional care.
- Don't use comparisons `>=` `>` `<` `<=` with a variable which may be `null/undefined`, unless you're really sure of what you're doing. If a variable can have these values, check for them separately.

Summary

- Comparison operators return a boolean value.
- Strings are compared letter-by-letter in the “dictionary” order.
- When values of different types are compared, they get converted to numbers (with the exclusion of a strict equality check).
- The values `null` and `undefined` equal `==` each other and do not equal any other value.
- Be careful when using comparisons like `>` or `<` with variables that can occasionally be `null/undefined`. Checking for `null/undefined` separately is a good idea.

✓ Tasks

Comparisons

importance: 5

What will be the result for these expressions?

```
5 > 4
"apple" > "pineapple"
"2" > "12"
undefined == null
undefined === null
null == "\n0\n"
null === +"\n0\n"
```

[To solution](#)

Conditional branching: if, '?'

Sometimes, we need to perform different actions based on different conditions.

To do that, we can use the `if` statement and the conditional operator `?`, that's also called a “question mark” operator.

The “if” statement

The `if(...)` statement evaluates a condition in parentheses and, if the result is `true`, executes a block of code.

For example:

```
let year = prompt('In which year was ECMAScript-2015 specification published?', '');
if (year == 2015) alert( 'You are right!' );
```

In the example above, the condition is a simple equality check (`year == 2015`), but it can be much more complex.

If we want to execute more than one statement, we have to wrap our code block inside curly braces:

```
if (year == 2015) {
  alert( "That's correct!" );
  alert( "You're so smart!" );
}
```

We recommend wrapping your code block with curly braces `{}` every time you use an `if` statement, even if there is only one statement to execute. Doing so improves readability.

Boolean conversion

The `if (...)` statement evaluates the expression in its parentheses and converts the result to a boolean.

Let's recall the conversion rules from the chapter [Type Conversions](#):

- A number `0`, an empty string `""`, `null`, `undefined`, and `NaN` all become `false`. Because of that they are called “falsy” values.

- Other values become `true`, so they are called “truthy”.

So, the code under this condition would never execute:

```
if (0) { // 0 is falsy
  ...
}
```

...and inside this condition – it always will:

```
if (1) { // 1 is truthy
  ...
}
```

We can also pass a pre-evaluated boolean value to `if`, like this:

```
let cond = (year == 2015); // equality evaluates to true or false

if (cond) {
  ...
}
```

The “else” clause

The `if` statement may contain an optional `else` block. It executes when the condition is falsy.

For example:

```
let year = prompt('In which year was the ECMAScript-2015 specification published?', '');

if (year == 2015) {
  alert( 'You guessed it right!' );
} else {
  alert( 'How can you be so wrong?' ); // any value except 2015
}
```

Several conditions: “else if”

Sometimes, we’d like to test several variants of a condition. The `else if` clause lets us do that.

For example:

```
let year = prompt('In which year was the ECMAScript-2015 specification published?', '');

if (year < 2015) {
  alert( 'Too early...' );
} else if (year > 2015) {
  alert( 'Too late' );
} else {
  alert( 'Exactly!' );
}
```

In the code above, JavaScript first checks `year < 2015`. If that is falsy, it goes to the next condition `year > 2015`. If that is also falsy, it shows the last `alert`.

There can be more `else if` blocks. The final `else` is optional.

Conditional operator ‘?’

Sometimes, we need to assign a variable depending on a condition.

For instance:

```

let accessAllowed;
let age = prompt('How old are you?', '');

if (age > 18) {
  accessAllowed = true;
} else {
  accessAllowed = false;
}

alert(accessAllowed);

```

The so-called “conditional” or “question mark” operator lets us do that in a shorter and simpler way.

The operator is represented by a question mark `?`. Sometimes it's called “ternary”, because the operator has three operands. It is actually the one and only operator in JavaScript which has that many.

The syntax is:

```
let result = condition ? value1 : value2;
```

The `condition` is evaluated: if it's truthy then `value1` is returned, otherwise – `value2`.

For example:

```
let accessAllowed = (age > 18) ? true : false;
```

Technically, we can omit the parentheses around `age > 18`. The question mark operator has a low precedence, so it executes after the comparison `>`.

This example will do the same thing as the previous one:

```

// the comparison operator "age > 18" executes first anyway
// (no need to wrap it into parentheses)
let accessAllowed = age > 18 ? true : false;

```

But parentheses make the code more readable, so we recommend using them.

Please note:

In the example above, you can avoid using the question mark operator because the comparison itself returns `true/false`:

```

// the same
let accessAllowed = age > 18;

```

Multiple ‘?’

A sequence of question mark operators `?` can return a value that depends on more than one condition.

For instance:

```

let age = prompt('age?', 18);

let message = (age < 3) ? 'Hi, baby!' :
  (age < 18) ? 'Hello!' :
  (age < 100) ? 'Greetings!' :
  'What an unusual age!';

alert( message );

```

It may be difficult at first to grasp what's going on. But after a closer look, we can see that it's just an ordinary sequence of tests:

1. The first question mark checks whether `age < 3`.
2. If true – it returns `'Hi, baby!'`. Otherwise, it continues to the expression after the colon “:”, checking `age < 18`.
3. If that's true – it returns `'Hello!'`. Otherwise, it continues to the expression after the next colon “:”, checking `age < 100`.
4. If that's true – it returns `'Greetings!'`. Otherwise, it continues to the expression after the last colon “:”, returning `'What an unusual age!'`.

Here's how this looks using `if...else`:

```
if (age < 3) {  
  message = 'Hi, baby!';  
} else if (age < 18) {  
  message = 'Hello!';  
} else if (age < 100) {  
  message = 'Greetings!';  
} else {  
  message = 'What an unusual age!';  
}
```

Non-traditional use of '?'

Sometimes the question mark `?` is used as a replacement for `if`:

```
let company = prompt('Which company created JavaScript?', '');  
  
(company == 'Netscape') ?  
  alert('Right!') : alert('Wrong.');
```

Depending on the condition `company == 'Netscape'`, either the first or the second expression after the `?` gets executed and shows an alert.

We don't assign a result to a variable here. Instead, we execute different code depending on the condition.

It's not recommended to use the question mark operator in this way.

The notation is shorter than the equivalent `if` statement, which appeals to some programmers. But it is less readable.

Here is the same code using `if` for comparison:

```
let company = prompt('Which company created JavaScript?', '');  
  
if (company == 'Netscape') {  
  alert('Right!');  
} else {  
  alert('Wrong.');
```

Our eyes scan the code vertically. Code blocks which span several lines are easier to understand than a long, horizontal instruction set.

The purpose of the question mark operator `?` is to return one value or another depending on its condition. Please use it for exactly that. Use `if` when you need to execute different branches of code.

✔ Tasks

if (a string with zero)

importance: 5

Will `alert` be shown?

```
if ("0") {  
  alert( 'Hello' );  
}
```

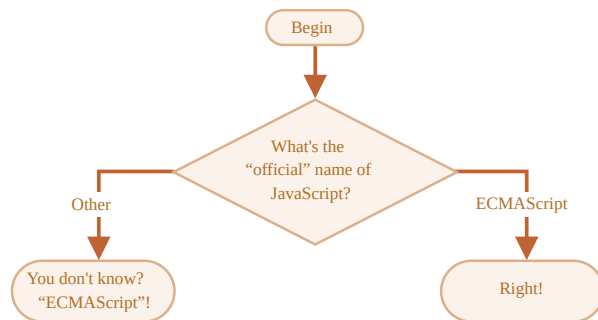
[To solution](#)

The name of JavaScript

importance: 2

Using the `if...else` construct, write the code which asks: 'What is the "official" name of JavaScript?'

If the visitor enters "ECMAScript", then output "Right!", otherwise – output: "You don't know? ECMAScript!"



[Demo in new window](#)

[To solution](#)

Show the sign

importance: 2

Using `if...else`, write the code which gets a number via `prompt` and then shows in `alert`:

- `1`, if the value is greater than zero,
- `-1`, if less than zero,
- `0`, if equals zero.

In this task we assume that the input is always a number.

[Demo in new window](#)

[To solution](#)

Rewrite 'if' into '?'

importance: 5

Rewrite this `if` using the conditional operator `'?'`:

```
let result;

if (a + b < 4) {
  result = 'Below';
} else {
  result = 'Over';
}
```

[To solution](#)

Rewrite 'if..else' into '?'

importance: 5

Rewrite `if...else` using multiple ternary operators `'?'`.

For readability, it's recommended to split the code into multiple lines.

```

let message;

if (login == 'Employee') {
  message = 'Hello';
} else if (login == 'Director') {
  message = 'Greetings';
} else if (login == '') {
  message = 'No login';
} else {
  message = '';
}

```

[To solution](#)

Logical operators

There are four logical operators in JavaScript: `||` (OR), `&&` (AND), `!` (NOT), `??` (Nullish Coalescing). Here we cover the first three, the `??` operator is in the next article.

Although they are called “logical”, they can be applied to values of any type, not only boolean. Their result can also be of any type.

Let’s see the details.

`||` (OR)

The “OR” operator is represented with two vertical line symbols:

```
result = a || b;
```

In classical programming, the logical OR is meant to manipulate boolean values only. If any of its arguments are `true`, it returns `true`, otherwise it returns `false`.

In JavaScript, the operator is a little bit trickier and more powerful. But first, let’s see what happens with boolean values.

There are four possible logical combinations:

```

alert( true || true ); // true
alert( false || true ); // true
alert( true || false ); // true
alert( false || false ); // false

```

As we can see, the result is always `true` except for the case when both operands are `false`.

If an operand is not a boolean, it’s converted to a boolean for the evaluation.

For instance, the number `1` is treated as `true`, the number `0` as `false`:

```

if (1 || 0) { // works just like if( true || false )
  alert( 'truthy!' );
}

```

Most of the time, OR `||` is used in an `if` statement to test if *any* of the given conditions is `true`.

For example:

```

let hour = 9;

if (hour < 10 || hour > 18) {
  alert( 'The office is closed.' );
}

```

We can pass more conditions:

```
let hour = 12;
let isWeekend = true;

if (hour < 10 || hour > 18 || isWeekend) {
  alert( 'The office is closed.' ); // it is the weekend
}
```

OR "||" finds the first truthy value

The logic described above is somewhat classical. Now, let's bring in the “extra” features of JavaScript.

The extended algorithm works as follows.

Given multiple OR'ed values:

```
result = value1 || value2 || value3;
```

The OR `||` operator does the following:

- Evaluates operands from left to right.
- For each operand, converts it to boolean. If the result is `true`, stops and returns the original value of that operand.
- If all operands have been evaluated (i.e. all were `false`), returns the last operand.

A value is returned in its original form, without the conversion.

In other words, a chain of OR `||` returns the first truthy value or the last one if no truthy value is found.

For instance:

```
alert( 1 || 0 ); // 1 (1 is truthy)

alert( null || 1 ); // 1 (1 is the first truthy value)
alert( null || 0 || 1 ); // 1 (the first truthy value)

alert( undefined || null || 0 ); // 0 (all falsy, returns the last value)
```

This leads to some interesting usage compared to a “pure, classical, boolean-only OR”.

1. Getting the first truthy value from a list of variables or expressions.

For instance, we have `firstName`, `lastName` and `nickName` variables, all optional (i.e. can be undefined or have falsy values).

Let's use OR `||` to choose the one that has the data and show it (or `"Anonymous"` if nothing set):

```
let firstName = "";
let lastName = "";
let nickName = "SuperCoder";

alert( firstName || lastName || nickName || "Anonymous" ); // SuperCoder
```

If all variables were falsy, `"Anonymous"` would show up.

2. Short-circuit evaluation.

Another feature of OR `||` operator is the so-called “short-circuit” evaluation.

It means that `||` processes its arguments until the first truthy value is reached, and then the value is returned immediately, without even touching the other argument.

The importance of this feature becomes obvious if an operand isn't just a value, but an expression with a side effect, such as a variable assignment or a function call.

In the example below, only the second message is printed:

```
true || alert("not printed");
false || alert("printed");
```

In the first line, the OR `||` operator stops the evaluation immediately upon seeing `true`, so the `alert` isn't run.

Sometimes, people use this feature to execute commands only if the condition on the left part is falsy.

&& (AND)

The AND operator is represented with two ampersands `&&`:

```
result = a && b;
```

In classical programming, AND returns `true` if both operands are truthy and `false` otherwise:

```
alert( true && true ); // true
alert( false && true ); // false
alert( true && false ); // false
alert( false && false ); // false
```

An example with `if`:

```
let hour = 12;
let minute = 30;

if (hour == 12 && minute == 30) {
  alert( 'The time is 12:30' );
}
```

Just as with OR, any value is allowed as an operand of AND:

```
if (1 && 0) { // evaluated as true && false
  alert( "won't work, because the result is falsy" );
}
```

AND “&&” finds the first falsy value

Given multiple AND'ed values:

```
result = value1 && value2 && value3;
```

The AND `&&` operator does the following:

- Evaluates operands from left to right.
- For each operand, converts it to a boolean. If the result is `false`, stops and returns the original value of that operand.
- If all operands have been evaluated (i.e. all were truthy), returns the last operand.

In other words, AND returns the first falsy value or the last value if none were found.

The rules above are similar to OR. The difference is that AND returns the first *falsy* value while OR returns the first *truthy* one.

Examples:

```
// if the first operand is truthy,
// AND returns the second operand:
alert( 1 && 0 ); // 0
alert( 1 && 5 ); // 5

// if the first operand is falsy,
// AND returns it. The second operand is ignored
alert( null && 5 ); // null
alert( 0 && "no matter what" ); // 0
```

We can also pass several values in a row. See how the first falsy one is returned:

```
alert( 1 && 2 && null && 3 ); // null
```

When all values are truthy, the last value is returned:

```
alert( 1 && 2 && 3 ); // 3, the last one
```

Precedence of AND && is higher than OR ||

The precedence of AND && operator is higher than OR ||.

So the code `a && b || c && d` is essentially the same as if the && expressions were in parentheses: `(a && b) || (c && d)`.

Don't replace if with || or &&

Sometimes, people use the AND && operator as a “shorter way to write if”.

For instance:

```
let x = 1;

(x > 0) && alert( 'Greater than zero!' );
```

The action in the right part of && would execute only if the evaluation reaches it. That is, only if `(x > 0)` is true.

So we basically have an analogue for:

```
let x = 1;

if (x > 0) alert( 'Greater than zero!' );
```

Although, the variant with && appears shorter, if is more obvious and tends to be a little bit more readable. So we recommend using every construct for its purpose: use if if we want if and use && if we want AND.

! (NOT)

The boolean NOT operator is represented with an exclamation sign !.

The syntax is pretty simple:

```
result = !value;
```

The operator accepts a single argument and does the following:

1. Converts the operand to boolean type: `true/false`.
2. Returns the inverse value.

For instance:

```
alert( !true ); // false
alert( !0 ); // true
```

A double NOT `!!` is sometimes used for converting a value to boolean type:

```
alert( !!"non-empty string" ); // true
alert( !!null ); // false
```

That is, the first NOT converts the value to boolean and returns the inverse, and the second NOT inverts it again. In the end, we have a plain value-to-boolean conversion.

There's a little more verbose way to do the same thing – a built-in `Boolean` function:

```
alert( Boolean("non-empty string") ); // true
alert( Boolean(null) ); // false
```

The precedence of NOT `!` is the highest of all logical operators, so it always executes first, before `&&` or `||`.

✔ Tasks

What's the result of OR?

importance: 5

What is the code below going to output?

```
alert( null || 2 || undefined );
```

[To solution](#)

What's the result of OR'ed alerts?

importance: 3

What will the code below output?

```
alert( alert(1) || 2 || alert(3) );
```

[To solution](#)

What is the result of AND?

importance: 5

What is this code going to show?

```
alert( 1 && null && 2 );
```

[To solution](#)

What is the result of AND'ed alerts?

importance: 3

What will this code show?

```
alert( alert(1) && alert(2) );
```

[To solution](#)

The result of OR AND OR

importance: 5

What will the result be?

```
alert( null || 2 && 3 || 4 );
```

[To solution](#)

Check the range between

importance: 3

Write an `if` condition to check that `age` is between `14` and `90` inclusively.

“Inclusively” means that `age` can reach the edges `14` or `90`.

[To solution](#)

Check the range outside

importance: 3

Write an `if` condition to check that `age` is NOT between `14` and `90` inclusively.

Create two variants: the first one using NOT `!`, the second one – without it.

[To solution](#)

A question about "if"

importance: 5

Which of these `alert`s are going to execute?

What will the results of the expressions be inside `if(...)`?

```
if (-1 || 0) alert( 'first' );
if (-1 && 0) alert( 'second' );
if (null || -1 && 1) alert( 'third' );
```

[To solution](#)

Check the login

importance: 3

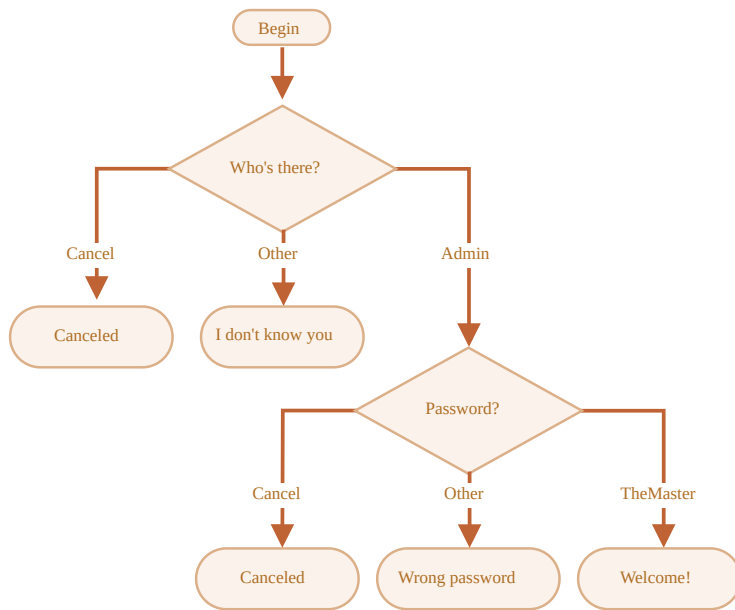
Write the code which asks for a login with `prompt`.

If the visitor enters `"Admin"`, then `prompt` for a password, if the input is an empty line or `Esc` – show “Canceled”, if it's another string – then show “I don't know you”.

The password is checked as follows:

- If it equals “TheMaster”, then show “Welcome!”,
- Another string – show “Wrong password”,
- For an empty string or cancelled input, show “Canceled”

The schema:



Please use nested `if` blocks. Mind the overall readability of the code.

Hint: passing an empty input to a prompt returns an empty string `''`. Pressing `ESC` during a prompt returns `null`.

[Run the demo](#)

[To solution](#)

Nullish coalescing operator '??'

⚠ A recent addition

This is a recent addition to the language. Old browsers may need [polyfills](#).

The nullish coalescing operator is written as two question marks `??`.

As it treats `null` and `undefined` similarly, we'll use a special term here, in this article. For brevity, we'll say that a value is "defined" when it's neither `null` nor `undefined`.

The result of `a ?? b` is:

- if `a` is defined, then `a`,
- if `a` isn't defined, then `b`.

In other words, `??` returns the first argument if it's not `null/undefined`. Otherwise, the second one.

The nullish coalescing operator isn't anything completely new. It's just a nice syntax to get the first "defined" value of the two.

We can rewrite `result = a ?? b` using the operators that we already know, like this:

```
result = (a !== null && a !== undefined) ? a : b;
```

Now it should be absolutely clear what `??` does. Let's see where it helps.

The common use case for `??` is to provide a default value.

For example, here we show `user` if its value isn't `null/undefined`, otherwise `Anonymous`:

```
let user;

alert(user ?? "Anonymous"); // Anonymous (user is undefined)
```

Here's the example with `user` assigned to a name:

```
let user = "John";

alert(user ?? "Anonymous"); // John (user is not null/undefined)
```

We can also use a sequence of `??` to select the first value from a list that isn't `null/undefined`.

Let's say we have a user's data in variables `firstName`, `lastName` or `nickName`. All of them may be not defined, if the user decided not to fill in the corresponding values.

We'd like to display the user name using one of these variables, or show "Anonymous" if all of them are `null/undefined`.

Let's use the `??` operator for that:

```
let firstName = null;
let lastName = null;
let nickName = "Supercoder";

// shows the first defined value:
alert(firstName ?? lastName ?? nickName ?? "Anonymous"); // Supercoder
```

Comparison with `||`

The OR `||` operator can be used in the same way as `??`, as it was described in the [previous chapter](#).

For example, in the code above we could replace `??` with `||` and still get the same result:

```
let firstName = null;
let lastName = null;
let nickName = "Supercoder";

// shows the first truthy value:
alert(firstName || lastName || nickName || "Anonymous"); // Supercoder
```

Historically, the OR `||` operator was there first. It's been there since the beginning of JavaScript, so developers were using it for such purposes for a long time.

On the other hand, the nullish coalescing operator `??` was added to JavaScript only recently, and the reason for that was that people weren't quite happy with `||`.

The important difference between them is that:

- `||` returns the first *truthy* value.
- `??` returns the first *defined* value.

In other words, `||` doesn't distinguish between `false`, `0`, an empty string `""` and `null/undefined`. They are all the same – falsy values. If any of these is the first argument of `||`, then we'll get the second argument as the result.

In practice though, we may want to use default value only when the variable is `null/undefined`. That is, when the value is really unknown/not set.

For example, consider this:

```
let height = 0;

alert(height || 100); // 100
alert(height ?? 100); // 0
```

- The `height || 100` checks `height` for being a falsy value, and it's `0`, falsy indeed.
 - so the result of `||` is the second argument, `100`.
- The `height ?? 100` checks `height` for being `null/undefined`, and it's not,
 - so the result is `height` "as is", that is `0`.

In practice, the zero height is often a valid value, that shouldn't be replaced with the default. So `??` does just the right thing.

Precedence

The precedence of the `??` operator is the same as `||`. They both equal 3 in the [MDN table](#).

That means that, just like `||`, the nullish coalescing operator `??` is evaluated before `=` and `?`, but after most other operations, such as `+`, `*`.

So we may need to add parentheses in expressions like this:

```
let height = null;
let width = null;

// important: use parentheses
let area = (height ?? 100) * (width ?? 50);

alert(area); // 5000
```

Otherwise, if we omit parentheses, then as `*` has the higher precedence than `??`, it would execute first, leading to incorrect results.

```
// without parentheses
let area = height ?? 100 * width ?? 50;

// ...works this way (not what we want):
let area = height ?? (100 * width) ?? 50;
```

Using `??` with `&&` or `||`

Due to safety reasons, JavaScript forbids using `??` together with `&&` and `||` operators, unless the precedence is explicitly specified with parentheses.

The code below triggers a syntax error:

```
let x = 1 && 2 ?? 3; // Syntax error
```

The limitation is surely debatable, it was added to the language specification with the purpose to avoid programming mistakes, when people start to switch from `||` to `??`.

Use explicit parentheses to work around it:

```
let x = (1 && 2) ?? 3; // Works

alert(x); // 2
```

Summary

- The nullish coalescing operator `??` provides a short way to choose the first “defined” value from a list.

It's used to assign default values to variables:

```
// set height=100, if height is null or undefined
height = height ?? 100;
```

- The operator `??` has a very low precedence, only a bit higher than `?` and `=`, so consider adding parentheses when using it in an expression.
- It's forbidden to use it with `||` or `&&` without explicit parentheses.

Loops: while and for

We often need to repeat actions.

For example, outputting goods from a list one after another or just running the same code for each number from 1 to 10.

Loops are a way to repeat the same code multiple times.

The for...of and for...in loops

A small announcement for advanced readers.

This article covers only basic loops: `while`, `do..while` and `for(..;...;..)`.

If you came to this article searching for other types of loops, here are the pointers:

- See [for...in](#) to loop over object properties.
- See [for...of](#) and [iterables](#) for looping over arrays and iterable objects.

Otherwise, please read on.

The “while” loop

The `while` loop has the following syntax:

```
while (condition) {  
  // code  
  // so-called "loop body"  
}
```

While the `condition` is truthy, the `code` from the loop body is executed.

For instance, the loop below outputs `i` while `i < 3`:

```
let i = 0;  
while (i < 3) { // shows 0, then 1, then 2  
  alert( i );  
  i++;  
}
```

A single execution of the loop body is called *an iteration*. The loop in the example above makes three iterations.

If `i++` was missing from the example above, the loop would repeat (in theory) forever. In practice, the browser provides ways to stop such loops, and in server-side JavaScript, we can kill the process.

Any expression or variable can be a loop condition, not just comparisons: the condition is evaluated and converted to a boolean by `while`.

For instance, a shorter way to write `while (i != 0)` is `while (i)`:

```
let i = 3;  
while (i) { // when i becomes 0, the condition becomes falsy, and the loop stops  
  alert( i );  
  i--;  
}
```

Curly braces are not required for a single-line body

If the loop body has a single statement, we can omit the curly braces `{...}`:

```
let i = 3;  
while (i) alert(i--);
```

The “do...while” loop

The condition check can be moved *below* the loop body using the `do..while` syntax:

```
do {  
  // loop body  
} while (condition);
```

The loop will first execute the body, then check the condition, and, while it's truthy, execute it again and again.

For example:

```
let i = 0;
do {
  alert( i );
  i++;
} while ( i < 3 );
```

This form of syntax should only be used when you want the body of the loop to execute **at least once** regardless of the condition being truthy. Usually, the other form is preferred: `while(...) {...}`.

The “for” loop

The `for` loop is more complex, but it's also the most commonly used loop.

It looks like this:

```
for (begin; condition; step) {
  // ... loop body ...
}
```

Let's learn the meaning of these parts by example. The loop below runs `alert(i)` for `i` from `0` up to (but not including) `3`:

```
for (let i = 0; i < 3; i++) { // shows 0, then 1, then 2
  alert(i);
}
```

Let's examine the `for` statement part-by-part:

part		
begin	<code>let i = 0</code>	Executes once upon entering the loop.
condition	<code>i < 3</code>	Checked before every loop iteration. If false, the loop stops.
body	<code>alert(i)</code>	Runs again and again while the condition is truthy.
step	<code>i++</code>	Executes after the body on each iteration.

The general loop algorithm works like this:

```
Run begin
→ (if condition → run body and run step)
→ (if condition → run body and run step)
→ (if condition → run body and run step)
→ ...
```

That is, `begin` executes once, and then it iterates: after each `condition` test, `body` and `step` are executed.

If you are new to loops, it could help to go back to the example and reproduce how it runs step-by-step on a piece of paper.

Here's exactly what happens in our case:

```
// for (let i = 0; i < 3; i++) alert(i)

// run begin
let i = 0
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
if (i < 3) { alert(i); i++ }
// if condition → run body and run step
```

```
if (i < 3) { alert(i); i++ }  
// ...finish, because now i == 3
```

Inline variable declaration

Here, the “counter” variable `i` is declared right in the loop. This is called an “inline” variable declaration. Such variables are visible only inside the loop.

```
for (let i = 0; i < 3; i++) {  
  alert(i); // 0, 1, 2  
}  
alert(i); // error, no such variable
```

Instead of defining a variable, we could use an existing one:

```
let i = 0;  
  
for (i = 0; i < 3; i++) { // use an existing variable  
  alert(i); // 0, 1, 2  
}  
  
alert(i); // 3, visible, because declared outside of the loop
```

Skipping parts

Any part of `for` can be skipped.

For example, we can omit `begin` if we don't need to do anything at the loop start.

Like here:

```
let i = 0; // we have i already declared and assigned  
  
for (; i < 3; i++) { // no need for "begin"  
  alert(i); // 0, 1, 2  
}
```

We can also remove the `step` part:

```
let i = 0;  
  
for (; i < 3;) {  
  alert(i++);  
}
```

This makes the loop identical to `while (i < 3)`.

We can actually remove everything, creating an infinite loop:

```
for (;;) {  
  // repeats without limits  
}
```

Please note that the two `for` semicolons `;` must be present. Otherwise, there would be a syntax error.

Breaking the loop

Normally, a loop exits when its condition becomes falsy.

But we can force the exit at any time using the special `break` directive.

For example, the loop below asks the user for a series of numbers, “breaking” when no number is entered:

```

let sum = 0;

while (true) {

    let value = +prompt("Enter a number", '');

    if (!value) break; // (*)

    sum += value;

}

alert( 'Sum: ' + sum );

```

The `break` directive is activated at the line `(*)` if the user enters an empty line or cancels the input. It stops the loop immediately, passing control to the first line after the loop. Namely, `alert`.

The combination “infinite loop + `break` as needed” is great for situations when a loop’s condition must be checked not in the beginning or end of the loop, but in the middle or even in several places of its body.

Continue to the next iteration

The `continue` directive is a “lighter version” of `break`. It doesn’t stop the whole loop. Instead, it stops the current iteration and forces the loop to start a new one (if the condition allows).

We can use it if we’re done with the current iteration and would like to move on to the next one.

The loop below uses `continue` to output only odd values:

```

for (let i = 0; i < 10; i++) {

    // if true, skip the remaining part of the body
    if (i % 2 == 0) continue;

    alert(i); // 1, then 3, 5, 7, 9

}

```

For even values of `i`, the `continue` directive stops executing the body and passes control to the next iteration of `for` (with the next number). So the `alert` is only called for odd values.

The `continue` directive helps decrease nesting

A loop that shows odd values could look like this:

```

for (let i = 0; i < 10; i++) {

    if (i % 2) {
        alert( i );
    }

}

```

From a technical point of view, this is identical to the example above. Surely, we can just wrap the code in an `if` block instead of using `continue`.

But as a side effect, this created one more level of nesting (the `alert` call inside the curly braces). If the code inside of `if` is longer than a few lines, that may decrease the overall readability.

⚠ No `break/continue` to the right of `'?`

Please note that syntax constructs that are not expressions cannot be used with the ternary operator `?`. In particular, directives such as `break/continue` aren't allowed there.

For example, if we take this code:

```
if (i > 5) {  
  alert(i);  
} else {  
  continue;  
}
```

...and rewrite it using a question mark:

```
(i > 5) ? alert(i) : continue; // continue isn't allowed here
```

...it stops working: there's a syntax error.

This is just another reason not to use the question mark operator `?` instead of `if`.

Labels for `break/continue`

Sometimes we need to break out from multiple nested loops at once.

For example, in the code below we loop over `i` and `j`, prompting for the coordinates `(i, j)` from `(0,0)` to `(2,2)`:

```
for (let i = 0; i < 3; i++) {  
  for (let j = 0; j < 3; j++) {  
    let input = prompt(`Value at coords (${i},${j})`, '');  
    // what if we want to exit from here to Done (below)?  
  }  
}  
  
alert('Done!');
```

We need a way to stop the process if the user cancels the input.

The ordinary `break` after `input` would only break the inner loop. That's not sufficient – labels, come to the rescue!

A *label* is an identifier with a colon before a loop:

```
labelName: for (...) {  
  ...  
}
```

The `break <labelName>` statement in the loop below breaks out to the label:

```
outer: for (let i = 0; i < 3; i++) {  
  for (let j = 0; j < 3; j++) {  
    let input = prompt(`Value at coords (${i},${j})`, '');  
    // if an empty string or canceled, then break out of both loops  
    if (!input) break outer; // (*)  
    // do something with the value...  
  }  
}  
  
alert('Done!');
```

In the code above, `break outer` looks upwards for the label named `outer` and breaks out of that loop.

So the control goes straight from `(*)` to `alert('Done!')`.

We can also move the label onto a separate line:

```
outer:
for (let i = 0; i < 3; i++) { ... }
```

The `continue` directive can also be used with a label. In this case, code execution jumps to the next iteration of the labeled loop.

⚠ Labels do not allow to “jump” anywhere

Labels do not allow us to jump into an arbitrary place in the code.

For example, it is impossible to do this:

```
break label; // jump to the label below (doesn't work)

label: for (...)
```

A `break` directive must be inside a code block. Technically, any labelled code block will do, e.g.:

```
label: {
  // ...
  break label; // works
  // ...
}
```

...Although, 99.9% of the time `break` is used inside loops, as we've seen in the examples above.

A `continue` is only possible from inside a loop.

Summary

We covered 3 types of loops:

- `while` – The condition is checked before each iteration.
- `do..while` – The condition is checked after each iteration.
- `for (;;)` – The condition is checked before each iteration, additional settings available.

To make an “infinite” loop, usually the `while(true)` construct is used. Such a loop, just like any other, can be stopped with the `break` directive.

If we don't want to do anything in the current iteration and would like to forward to the next one, we can use the `continue` directive.

`break/continue` support labels before the loop. A label is the only way for `break/continue` to escape a nested loop to go to an outer one.

✓ Tasks

Last loop value

importance: 3

What is the last value alerted by this code? Why?

```
let i = 3;

while (i) {
  alert( i-- );
}
```

[To solution](#)

Which values does the while loop show?

importance: 4

For every loop iteration, write down which value it outputs and then compare it with the solution.

Both loops `alert` the same values, or not?

1.

The prefix form `++i`:

```
let i = 0;
while (++i < 5) alert( i );
```

2.

The postfix form `i++`:

```
let i = 0;
while (i++ < 5) alert( i );
```

[To solution](#)

Which values get shown by the "for" loop?

importance: 4

For each loop write down which values it is going to show. Then compare with the answer.

Both loops `alert` same values or not?

1.

The postfix form:

```
for (let i = 0; i < 5; i++) alert( i );
```

2.

The prefix form:

```
for (let i = 0; i < 5; ++i) alert( i );
```

[To solution](#)

Output even numbers in the loop

importance: 5

Use the `for` loop to output even numbers from 2 to 10.

[Run the demo](#)

[To solution](#)

Replace "for" with "while"

importance: 5

Rewrite the code changing the `for` loop to `while` without altering its behavior (the output should stay same).

```
for (let i = 0; i < 3; i++) {  
  alert( `number ${i}!` );  
}
```

[To solution](#)

Repeat until the input is correct

importance: 5

Write a loop which prompts for a number greater than `100`. If the visitor enters another number – ask them to input again.

The loop must ask for a number until either the visitor enters a number greater than `100` or cancels the input/enters an empty line.

Here we can assume that the visitor only inputs numbers. There's no need to implement a special handling for a non-numeric input in this task.

[Run the demo](#)

[To solution](#)

Output prime numbers

importance: 3

An integer number greater than `1` is called a [prime](#) if it cannot be divided without a remainder by anything except `1` and itself.

In other words, `n > 1` is a prime if it can't be evenly divided by anything except `1` and `n`.

For example, `5` is a prime, because it cannot be divided without a remainder by `2`, `3` and `4`.

Write the code which outputs prime numbers in the interval from `2` to `n`.

For `n = 10` the result will be `2, 3, 5, 7`.

P.S. The code should work for any `n`, not be hard-tuned for any fixed value.

[To solution](#)

The "switch" statement

A `switch` statement can replace multiple `if` checks.

It gives a more descriptive way to compare a value with multiple variants.

The syntax

The `switch` has one or more `case` blocks and an optional default.

It looks like this:

```
switch(x) {  
  case 'value1': // if (x === 'value1')  
    ...  
    [break]  
  
  case 'value2': // if (x === 'value2')  
    ...  
    [break]  
  
  default:  
    ...  
}
```

```
[break]
}
```

- The value of `x` is checked for a strict equality to the value from the first `case` (that is, `value1`) then to the second (`value2`) and so on.
- If the equality is found, `switch` starts to execute the code starting from the corresponding `case`, until the nearest `break` (or until the end of `switch`).
- If no case is matched then the `default` code is executed (if it exists).

An example

An example of `switch` (the executed code is highlighted):

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
    break;
  case 4:
    alert( 'Exactly!' );
    break;
  case 5:
    alert( 'Too big' );
    break;
  default:
    alert( "I don't know such values" );
}
```

Here the `switch` starts to compare `a` from the first `case` variant that is `3`. The match fails.

Then `4`. That's a match, so the execution starts from `case 4` until the nearest `break`.

If there is no `break` then the execution continues with the next `case` without any checks.

An example without `break`:

```
let a = 2 + 2;

switch (a) {
  case 3:
    alert( 'Too small' );
  case 4:
    alert( 'Exactly!' );
  case 5:
    alert( 'Too big' );
  default:
    alert( "I don't know such values" );
}
```

In the example above we'll see sequential execution of three `alert`s:

```
alert( 'Exactly!' );
alert( 'Too big' );
alert( "I don't know such values" );
```

i Any expression can be a `switch/case` argument

Both `switch` and `case` allow arbitrary expressions.

For example:

```
let a = "1";
let b = 0;

switch (+a) {
  case b + 1:
    alert("this runs, because +a is 1, exactly equals b+1");
    break;

  default:
    alert("this doesn't run");
}
```

Here `+a` gives `1`, that's compared with `b + 1` in `case`, and the corresponding code is executed.

Grouping of “case”

Several variants of `case` which share the same code can be grouped.

For example, if we want the same code to run for `case 3` and `case 5`:

```
let a = 3;

switch (a) {
  case 4:
    alert('Right!');
    break;

  case 3: // (*) grouped two cases
  case 5:
    alert('Wrong!');
    alert("Why don't you take a math class?");
    break;

  default:
    alert('The result is strange. Really.');
```

Now both `3` and `5` show the same message.

The ability to “group” cases is a side effect of how `switch/case` works without `break`. Here the execution of `case 3` starts from the line `(*)` and goes through `case 5`, because there's no `break`.

Type matters

Let's emphasize that the equality check is always strict. The values must be of the same type to match.

For example, let's consider the code:

```
let arg = prompt("Enter a value?");
switch (arg) {
  case '0':
  case '1':
    alert('One or zero');
    break;

  case '2':
    alert('Two');
    break;

  case 3:
    alert('Never executes!');
    break;

  default:
```

```
    alert( 'An unknown value' );  
}
```

1. For `0`, `1`, the first `alert` runs.
2. For `2` the second `alert` runs.
3. But for `3`, the result of the `prompt` is a string `"3"`, which is not strictly equal `===` to the number `3`. So we've got a dead code in `case 3`! The `default` variant will execute.

✓ Tasks

Rewrite the "switch" into an "if"

importance: 5

Write the code using `if..else` which would correspond to the following `switch`:

```
switch (browser) {  
  case 'Edge':  
    alert( "You've got the Edge!" );  
    break;  
  
  case 'Chrome':  
  case 'Firefox':  
  case 'Safari':  
  case 'Opera':  
    alert( 'Okay we support these browsers too' );  
    break;  
  
  default:  
    alert( 'We hope that this page looks ok!' );  
}
```

[To solution](#)

Rewrite "if" into "switch"

importance: 4

Rewrite the code below using a single `switch` statement:

```
let a = +prompt('a?', '');  
  
if (a == 0) {  
  alert( 0 );  
}  
if (a == 1) {  
  alert( 1 );  
}  
  
if (a == 2 || a == 3) {  
  alert( '2,3' );  
}
```

[To solution](#)

— ..